

CricSense AI

Natural-Language IPL Analytics Platform

Text-to-SQL · AI insights · Full-stack product · PM roadmap

A SQL coursework analysis of 16 IPL seasons, rebuilt as a live, AI-powered product: a conversational analytics engine where anyone can query ball-by-ball cricket data in plain English — with a guardrailed text-to-SQL core, auto-generated charts, and a complete product story.

Stack	FastAPI · React · DuckDB/Postgres · Claude (LLM)
Data	~260,000 deliveries · 16 IPL seasons (2008–2024)
Core feature	Guardrailed natural-language → SQL
Deliverables	GitHub repo · Live site (Lovable) · PRD & roadmap
Author	Nimish Vohra · nimishvohra576@gmail.com

1 · Problem & the pivot

The original project was a static slide deck of 25+ SQL queries analyzing IPL trends — attacking batsmen, anchors, hard-hitters, economy bowlers, all-rounders, wicket-keepers, and venue scoring. It was solid analysis, but the insight was locked behind SQL and frozen into fixed tables. A reader couldn't ask a new, specific question.

CricSense AI reframes that work as a **product**: the same analytical logic becomes a live engine where scouts, fantasy players, journalists, and fans type questions in plain English and get SQL-backed answers, charts, and a one-line scouting note — in seconds, with the generated SQL shown for trust.

What changed

Before	After
Static deck of 25+ SQL queries	Live web app, queryable in English
Reader sees fixed tables only	User asks novel, specific questions
Insight requires SQL knowledge	LLM writes & runs safe SQL for them
No product framing or audience	Personas, success metrics, roadmap

2 · What it does

- **Text-to-SQL**: ask in English; an LLM generates a single safe SELECT, which is validated and executed against the warehouse.
- **Auto-visualization**: every answer returns the most appropriate chart (bar, line, scatter) plus a result table.
- **Player archetypes**: seven pre-built analytical lenses — attacking batsmen, anchors, hard-hitters, economy bowlers, wicket-takers, all-rounders, keepers.
- **Venue intelligence**: run-scoring trends and pitch behaviour by stadium and season.
- **Guardrailed SQL**: read-only execution, allow-listing, AST validation, statement timeouts — injection-proof by design.
- **Insight narration**: an LLM summarizes each result into a concise scouting note.

3 · Architecture

Three tiers: a React SPA, a FastAPI service, and a SQL warehouse. The request lifecycle for a question:

- User submits a question to POST /api/ask.
- **Text-to-SQL (LLM)**: question + a schema-scoped prompt → a single SELECT.
- **Guardrail**: the SQL is parsed with sqlglot; non-SELECT statements, stacked queries, writes/DDL, and unknown tables are rejected; a row LIMIT is appended.
- **Execution**: runs on a read-only connection with a statement timeout.
- **Chart + narrative (LLM)**: a cheap call picks the chart type; another writes a 2–3 sentence insight.
- The SPA renders narrative, chart, table, and a collapsible “view SQL” panel.

Key design decisions

DuckDB demo, Postgres prod	Same ANSI SQL in both; zero-setup clone-and-run demo.
Curated v_* views	The original 25+ queries become tested, first-class views the LLM is steered toward.
Guardrail as its own layer	Defense-in-depth on top of a read-only role; clear, user-facing rejections.
LLM picks the chart	Removes a decision from the user; the result “just shows up” right.

4 • How text-to-SQL stays safe

The most interview-worthy part of the project. “What stops the LLM from dropping your table?” is answered in layers:

1	Schema-scoped prompt	Model sees only table/column names + allowed views — no write verbs.
2	AST validation (sqlglot)	SQL is parsed to a tree; anything not a single SELECT is rejected.
3	Table allow-list	Any reference outside the known schema — even in a subquery — is blocked.
4	Read-only DB role	The execution role has no INSERT/UPDATE/DELETE/DDL grants.
5	Timeout + row cap	Runaway scans are killed; result payloads are LIMIT-ed.

Why parse instead of regex?

Regex blocklists are bypassable (comments, casing, nested queries). Parsing to an AST and walking the tree is robust — a DROP hidden in a subquery is still a Drop node and is rejected.

Examples

```
SELECT 1; DROP TABLE ipl_ball
→ rejected: only a single SELECT is allowed

DELETE FROM ipl_ball
→ rejected: Delete statement not allowed

SELECT * FROM pg_user
→ rejected: table not permitted

SELECT * FROM v_attacking_batsman
→ accepted, capped to 100 rows
```

5 • Insights the platform surfaces

Drawn from the original 16-season analysis, now queryable on demand:

Metric	Leader	Value
Strike rate (500+ balls)	AD Russell	182.3
Boundary % (hard-hitter)	SP Narine	81.2%
Most runs	DA Warner	10,508
Economy (500+ balls)	Sohail Tanvir	6.25 RPO
Bowling strike rate	Sohail Tanvir	12.05
Highest-scoring venue	Eden Gardens	23,658 runs

Most common dismissal	Caught	11,486
-----------------------	--------	--------

6 · Product thinking

Personas & jobs-to-be-done

- **Fantasy player:** find undervalued hard-hitters before the auction.
- **Club scout:** spot cheap all-rounders with strong bowling strike rates.
- **Sports journalist:** get a defensible stat fast — with the SQL shown.
- **Casual fan:** settle a GOAT debate with real numbers.

Success metrics

North-star	Weekly Answered Questions (WAQ)
Answer acceptance rate	> 90%
p95 question→answer latency	< 6s
Sessions with 3+ questions	> 40%
Guardrail false-reject rate	< 5%

Roadmap

- **NOW · v1 (shipped):** guardrailed text-to-SQL, 7 archetypes, venue intelligence, charts + AI narration.
- **NEXT · v2:** win-probability model, fantasy-team optimizer, saved & shareable answers, player comparison.
- **LATER · v3:** multi-league (BBL, PSL), real-time ingestion, custom alerts, mobile app.

7 · Deliverables & how to run

- **GitHub repo** — backend (FastAPI + guardrail + LLM services), SQL views, React frontend, product docs, CI, tests.
- **Live website** — generated via the included Lovable prompt; speaks the same API.
- **This document + slide deck** — portfolio and presentation artifacts.

Quickstart

```
cd backend && pip install -r requirements.txt
cp .env.example .env # add ANTHROPIC_API_KEY
python -m app.seed # builds DuckDB warehouse (synthetic if no CSVs)
uvicorn app.main:app --reload

cd ../frontend && npm install && npm run dev
```

Original SQL analysis & concept by Nimish Vohra. Productized into CricSense AI as a full-stack, AI-powered analytics platform. Data: public IPL ball-by-ball dataset (2008–2024).